

Глава 17

Работа с числами

В главе описываются компоненты стандартной библиотеки C++, предназначенные для работы с числами, в частности, компоненты для работы со случайными числами и распределениями, классы комплексных чисел, глобальные численные функции, унаследованные от библиотеки C, и массивы значений.

Но кроме указанных выше инструментов стандартная библиотека C++ содержит и другие компоненты для работы с числами.

1. Для всех основных числовых типов определены свойства `numeric_limits`, зависящие от реализации (см. раздел 5.3).
2. Класс `ratio<>` поддерживает дробную арифметику, главным образом в качестве основы для работы со временем (см. раздел 5.6).
3. Библиотека STL содержит некоторые числовые алгоритмы, описанные в разделе 11.11.
4. Стандартная библиотека C++ содержит класс `valarray` для работы с массивами чисел. Однако на практике этот класс почти не используется, поэтому он лишь кратко упоминается в разделе 17.4, а его полное описание приводится в приложении, доступном по адресу <http://www.cppstdlib.com>.

17.1. Случайные числа и распределения

После принятия стандарта C++11 в стандартную библиотеку C++ включена библиотека для работы со случайными числами и распределениями, состоящая из большого набора классов и типов, способных удовлетворить потребности как новичков, так и экспертов. Она более сложная, чем может ожидать наивный программист. Разумеется, она содержит многие хорошо известные *распределения*. Но помимо этого стандартная библиотека C++ содержит много *генераторов случайных чисел* (engines). Эти генераторы порождают случайные значения без знака, равномерно распределенные между заданными минимумом и максимумом, а распределения преобразуют эти значения в случайные числа, линейно или нелинейно распределенные в соответствии с параметрами, заданными пользователем¹. Таким образом, значения, порожденные генераторами, не следует использовать непосредственно (подробности изложены в разделе 17.1.1). Библиотека содержит много генераторов, потому что никакие числа в компьютере не являются истинно случайными и приходится прилагать много усилий, для того чтобы добиться высокой степени случайности, так что генераторы отличаются друг от друга по качеству, размеру и скорости работы.

¹ С формальной точки зрения результат работы функции распределения правильнее называть *псевдослучайными*, а не *случайными числами*.

Отметим, что термин *генератор псевдослучайных чисел* является довольно двусмысленным, потому что он применяется к двум сущностям.

1. Он может относиться к источнику случайности. В соответствии со стандартом каждый генератор должен соответствовать требованиям, предъявляемым к генератору равномерно распределенных случайных чисел.
2. Он может относиться к механизму для генерации случайных чисел, представляющему собой генератор равномерно распределенных чисел и распределение.

Говоря о генераторе случайных чисел в программе, мы чаще всего будем иметь в виду второе утверждение. Таким образом, для работы нам необходимы генератор и распределение, что, как показано в первом примере, не вызывает затруднений.

Для использования библиотеки случайных чисел в программу следует включить заголовочный файл `<random>`.

17.1.1. Первый пример

Прежде чем перейти к обсуждению деталей, рассмотрим первый пример, в котором случайные числа используются для перетасовки элементов. Качество случайных чисел и перетасовки в данном случае не имеет значения.

```
// num/random1.cpp

#include <random>
#include <iostream>
#include <algorithm>
#include <vector>

int main()
{
    // создаем генератор случайных чисел по умолчанию
    std::default_random_engine dre;

    // используем генератор для порождения целых чисел
    // в интервале от 10 до 20 (включая оба эти числа)
    std::uniform_int_distribution<int> di(10,20);
    for (int i=0; i<20; ++i) {
        std::cout << di(dre) << " ";
    }
    std::cout << std::endl;

    // используем генератор для порождения чисел с плавающей точкой
    // в интервале от 10.0 до 20.0 (включая 10.0 и исключая 20.0)
    std::uniform_real_distribution<double> dr(10,20);
    for (int i=0; i<8; ++i) {
        std::cout << dr(dre) << " ";
    }
    std::cout << std::endl;

    // используем генератор для перетасовки элементов
    std::vector<int> v = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    ...
}
```

```

std::shuffle (v.begin(), v.end(), // интервал
             dre);                // источник случайности
for (int i=0; i<v.size(); ++i) {
    std::cout << v[i] << " ";
}
std::cout << std::endl;
}

```

Эта программа демонстрирует общий подход к библиотеке случайных чисел. После включения заголовочного файла для библиотеки случайных чисел

```
#include <random>
```

мы создаем случайные числа, комбинируя *генератор* `dre` с двумя *распределениями* `di` и `dr`.

- **Генераторы случайных чисел** являются источником случайности, имеющим состояние (stateful source of randomness). Это функциональные объекты, способные генерировать равномерно распределенные случайные значения без знака в интервале, заданном минимальным и максимальным значениями.
- **Распределения** позволяют указать, как на основе этих случайных значений создать случайные числа, распределенные на заданном интервале в соответствии с параметрами, заданными пользователем.

Стандартная библиотека C++ содержит много генераторов случайных чисел, потому что существует много подходов к их созданию (алгоритмы и их реализации различаются как по качеству, так и по быстродействию). В данном случае мы используем генератор, заданный по умолчанию, поскольку наша задача “обычная, простая и/или не требует особой точности”.

```
std::default_random_engine dre;
```

В соответствии со стандартом такой генератор можно назвать “приемлемым ...с точки зрения быстродействия, размера, качества или сочетания этих факторов”. Тем не менее выбор генератора по умолчанию зависит от реализации. По этой причине результаты работы программы на разных платформах могут оказаться разными.

Стандартная библиотека C++ содержит много распределений для разных типов числовых данных: равномерное, нормальное/гауссовское, экспоненциальное, гамма, Бернулли и др. Наиболее часто используемым является равномерное распределение, которое предполагает, что числа равномерно распределены по интервалу, заданному минимальным и максимальным значениями. Для этого распределения предусмотрены два класса: один для целых чисел, другой для действительных.

Сначала мы используем класс `uniform_int_distribution`, генерирующий целые числа. Эти числа могут иметь тип `short`, `int`, `long`, `long long` и соответствующие им типы без знаков. Если тип не указан, по умолчанию используется тип `int`. Первый аргумент конструктора задает минимальное значение (по умолчанию: 0), а второй — максимальное значение (по умолчанию: `numeric_limits<min>::max()`; см. раздел 5.3). Отметим, что минимальное и максимальное числа можно генерировать, поэтому интервал *не является* полуоткрытым. Следовательно, следующий оператор определяет, что случайные числа, созданные генератором `di`, распределены между значениями 10 и 20 (включая оба):

```
std::uniform_int_distribution<int> di(10,20);
```

Для генерации чисел в классе распределения предусмотрена операторная функция `()`, получающая генератор как аргумент. Таким образом, для создания следующего случайного числа, соответствующего этому распределению, следует вызвать конструктор

```
di(dre)
```

Отметим, что начальное состояние генератора точно задано и не является случайным. Таким образом, следующий код дважды выведет одно и то же значение:

```
std::uniform_int_distribution<int> d;
std::default_random_engine dre1;
std::cout << d(dre1) << " ";
std::default_random_engine dre2;
std::cout << d(dre2) << " ";
```

Если требуется непредсказуемое случайное число, необходимо случайным образом задать состояние генератора с помощью параметра, на который программа не может влиять, например, количество миллисекунд между двумя щелчками мышью. Иначе говоря, необходимо задать *начальное значение* (seed) для конструктора генератора. Рассмотрим пример:

```
unsigned int seed = ...           // задаем действительно случайное значение
std::default_random_engine dre(seed); // и используем его как начальное
                                     // состояние генератора
```

Кроме того, с помощью *начального значения* можно изменить состояние уже существующего генератора, используя функцию-член.

Точно так же генерируются числа с плавающей точкой. Однако в этом случае минимум и максимум действительно определяют полуоткрытый интервал. По умолчанию интервал имеет вид $[0.0, 1.0)$. Возможны следующие типы: `float`, `double` или `long double`, причем `double` — тип, используемый по умолчанию. Таким образом, следующий оператор определяет распределение `dr` и генерирует значение в интервале от 10.0 до 20.0 (исключая 20.0).

```
std::uniform_real_distribution<double> dr(10,20);
```

В противоположность этому следующая инструкция заставляет объект `d` генерировать значения типа `double` в интервале от 0.0 до 0.9999..., т.е. в интервале, не содержащем число 1.0:

```
std::uniform_real_distribution<> d;
```

В заключение программа демонстрирует, как с помощью генератора случайных чисел перетасовать элементы в контейнере или интервале. Начиная со стандарта C++11 в библиотеку включен алгоритм `shuffle()` (см. раздел 11.8.4), использующий генератор равномерно распределенных случайных чисел, такой как `std::default_random_engine`, для перетасовки элементов.

```
std::default_random_engine dre;
...
std::shuffle (v.begin(), v.end(), // интервал
             dre);                // источник случайности
```


Результат работы программы может выглядеть следующим образом²:

```
10 11 18 15 15 12 10 17 17 20 14 15 19 10 10 15 17 10 14 10
16.8677 19.3044 15.2693 16.5392 17.0119 17.622 10.4746 13.2823
1 6 3 4 2 8 9 5 7
```

Будьте осторожны, работая с временными генераторами

Программист может, но не должен передавать в качестве аргумента только что созданный временный генератор³. Причина заключается в том, что при каждой инициализации генератора его начальное состояние оказывается одним и тем же. Таким образом, если программа содержит инструкции, приведенные ниже, то дважды будет выполнена одна и та же перетасовка.

```
std::shuffle(v.begin(),v.end(),          // интервал
             std::default_random_engine()); // генератор случайных чисел
...
std::shuffle(v.begin(),v.end(),          // интервал
             std::default_random_engine()); // генератор случайных чисел
```

Другими словами, если первый элемент в первый раз был переставлен в конец интервала, то в следующий раз первый элемент снова будет переставлен в конец интервала. Таким образом, при каждом вызове перетасовка модифицирует позиции каждого элемента одним и тем же способом. Рассмотрим пример:

```
1 2 3 4 5 6 7 8 9 // начальное состояние
8 7 5 6 2 4 9 3 1 // после первой перетасовки
3 9 2 4 7 6 1 5 8 // после второй перетасовки с помощью
                  // генератора с неизменным состоянием
```

Этот пример показывает, что перетасовка может создать проблемы, не говоря уже о том, что ее результат после первого вызова становится предсказуемым. Независимо от того, насколько часто вызывается алгоритм `shuffle()`, число 4 всегда будет занимать только четвертую или шестую позицию соответственно текущему состоянию, а алгоритм `shuffle()` просто переставляет местами эти две позиции.

Для того чтобы избежать этой ловушки, следует использовать следующий код, в котором перетасовка каждый раз производится с новым состоянием:

```
std::default_random_engine dre;
...
std::shuffle(v.begin(),v.end(), // интервал
             dre);              // генератор случайных чисел
...
std::shuffle(v.begin(),v.end(), // интервал
             dre);              // генератор случайных чисел
```

²Здесь использовано слово “может”, потому что точное определение генератора `default_random_engine` зависит от реализации. В данном случае в качестве генератора `default_random_engine` используется класс `minstd_rand0`.

³Благодарю за это замечание Уолтера Э. Брауна (Walter E. Brown).

Теперь каждая перетасовка отличается от другой, и результат может выглядеть следующим образом:

```
1 2 3 4 5 6 7 8 9 // начальное состояние
8 7 5 6 2 4 9 3 1 // после первой перетасовки
3 6 2 7 1 5 8 9 4 // после второй перетасовки с помощью генератора,
// имеющего разные состояния
```

Не используйте генераторы без распределений

Наивный программист может спросить: почему просто не использовать значения, созданные генератором, в качестве случайных чисел, а если интервал не совпадет с требуемым, просто использовать деление по модулю %?

Ответ на этот важный вопрос дан в разделе 7.4.4 книги Andrew Koenig и Barbara E. Moo *Accelerated C++* (см. [KoenigMoo:Accelerated])⁴. Авторы этой книги объясняют, почему использование стандартного генератора случайных чисел `rand()` из языка C порождает проблемы. Существо дела состоит в следующем (точные цитаты выделены курсивом).

Вычисление случайных чисел с помощью операции `rand() % n` на практике является ошибочным по двум причинам.

1. Если n — небольшое целое число, то многие реализации генераторов случайных чисел выдают остатки, которые не являются в достаточной степени случайными. Например, довольно часто последовательные результаты функции `rand()` состоят из чередующихся четных и нечетных чисел. В этом случае, если n равно 2, последовательные результаты операции `rand() % n` будут принимать чередующиеся значения 0 и 1.
2. С другой стороны, если число n велико, а максимальное генерируемое значение не делится на n нацело, то некоторые остатки будут появляться чаще других. Например, если максимум равен 32767, а n равно 2000, то 17 сгенерированных значений (500, 2500, ..., 30500, 32500) будут преобразованы в 500 и только 16 сгенерированных значений (1500, 3500, ..., 31500) будут отображены в 1500. Причем, чем больше n , тем хуже становится ситуация.

Хорошие линейные генераторы учитывают эти ситуации при отображении сгенерированных значений в целевой интервал случайных чисел. Таким образом, для того чтобы получить случайные числа хорошего качества, генераторы и распределения всегда следует использовать совместно.

17.1.2. Генераторы

Стандартная библиотека C++ содержит 16 генераторов случайных чисел, которые можно использовать для вычисления случайных чисел, комбинируя их с распределениями или перетасовками (рис. 17.1).

Как указывалось выше, генератор случайных чисел — это *источник случайности, имеющий состояние*. Это состояние определяет последовательность генерируемых случайных значений, но не случайных чисел (см. раздел 17.1.1). При каждом вызове функции с помощью

⁴ Русский перевод: Кениг Э., Му Б. *Эффективное программирование на C++*. — М.: Вильямс, 2002. — 382 с. — Примеч. ред.

оператора `()` возникает новое случайное значение без знака, а внутреннее состояние генератора изменяется, чтобы в следующий раз появилось новое случайное значение.

Переходы между состояниями и генерируемые значения обычно точно определены. Таким образом, при одном и том же состоянии конкретный генератор на любой платформе создаст одно и то же случайное значение. Единственным исключением является генератор `default_random_engine`, зависящий от реализации. Он вычисляет предсказуемые значения, но алгоритмы, лежащие в его основе, на разных платформах могут быть разными.

Можно, конечно, сказать, что каждое значение, созданное генератором, всегда должно быть случайным, но на компьютере ни одно число не бывает истинно случайным. Если требуется действительно непредсказуемое значение, состояние генератора следует обрабатывать случайным образом, не зависящим от программы, например, определяя количество миллисекунд между двумя последовательными щелчками мышью.

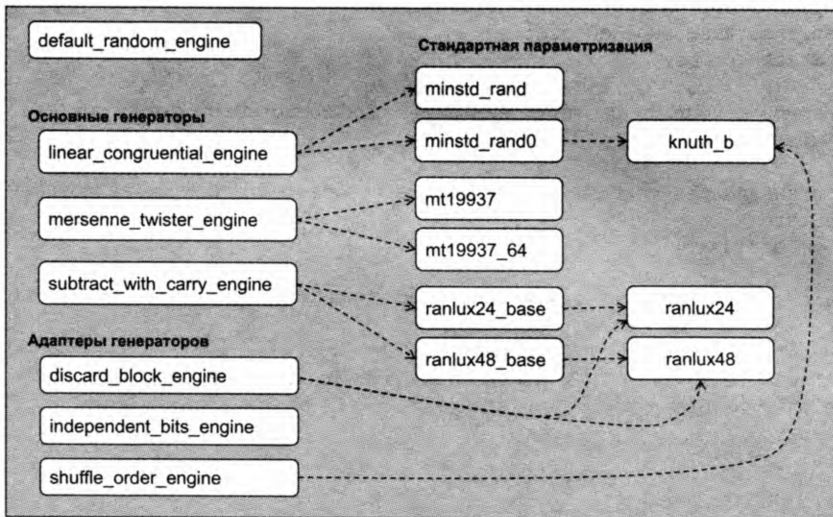


Рис. 17.1. Стандартные генераторы псевдослучайных чисел

С другой стороны, предсказуемость случайных величин дает преимущество: можно создавать одинаковые “случайные” сценарии для тестирования на основе случайных чисел.

Эту возможность демонстрирует следующая программа:

```
// num/random2.cpp

#include <random>
#include <iostream>
#include <sstream>

void printNumbers (std::default_random_engine& dre)
{
    for (int i=0; i<6; ++i) {
        std::cout << dre() << " ";
    }
    std::cout << std::endl;
}
```

```
int main()
{
    // создаем генератор и генерируем случайные числа
    std::default_random_engine dre;
    printNumbers(dre);

    // создаем аналогичный генератор и генерируем числа
    std::default_random_engine dre2; // начальное состояние, как у генератора dre
    printNumbers(dre2);

    // создаем генератор с начальным состоянием, заданным начальным значением
    std::default_random_engine dre3(42);
    printNumbers(dre3);

    // сохранить состояние генератора dre
    std::stringstream engineState;
    engineState << dre;

    // генерируем числа в соответствии с текущим состоянием генератора dre
    printNumbers(dre);

    // восстанавливаем сохраненное состояние генератора dre и снова создаем числа
    engineState >> dre;
    printNumbers(dre);

    // восстанавливаем сохраненное состояние генератора dre, пропускаем 2 числа
    // и снова создаем числа
    engineState.clear(); // создаем бит EOF
    engineState.seekg(0); // и перемещаем позицию чтения в начало
    engineState >> dre;
    dre.discard(2);
    printNumbers(dre);
}
```

Программа может вывести следующий результат⁵:

```
16807 282475249 1622650073 984943658 1144108930 470211272
16807 282475249 1622650073 984943658 1144108930 470211272
705894 1126542223 1579310009 565444343 807934826 421520601
101027544 1457850878 1458777923 2007237709 823564440 1115438165
101027544 1457850878 1458777923 2007237709 823564440 1115438165
1458777923 2007237709 823564440 1115438165 1784484492 74243042
```

После объявления генератора случайных чисел (в нашем случае — заданного по умолчанию) для создания случайных значений можно выполнить оператор `()`.

```
std::default_random_engine dre;
...
dre()
```

Таким образом, генератор — это функциональный объект (т.е. объект, который ведет себя как функция; см. раздел 6.10).

⁵Точный вывод будет другим, потому что генератор `default_random_engine` зависит от реализации. Здесь значения соответствуют определению генератора `default_random_engine` в виде `minstd_rand0`.

Генерируются целые значения без знака. Обычно можно задать точный тип, за исключением `default_random_engine`, который зависит от реализации. Для каждого генератора операция `result_type` определяет тип, а статические функции-члены `min()` и `max()` возвращают минимальные и максимальные генерируемые значения (в интервал включаются оба).

Если снова объявить такой же генератор, он будет иметь такое же начальное состояние и будет создавать те же самые случайные значения. Для того чтобы получить то же самое значение, можно дважды передать генератор функции `printNumbers()` по значению. Поскольку функция `printNumbers()` при каждом вызове будет работать с временной копией генератора `dre`, будут дважды созданы одни и те же случайные числа.

Следующий оператор показывает, как передать начальное значение, чтобы изменить начальное состояние генератора:

```
std::default_random_engine dre3(42);
```

Обычно, чтобы получить действительно случайные числа, в качестве начального значения необходимо передать значение, поступающее извне программы (например, интервал времени между двумя щелчками мышью или значение, выданное таймером).

Следующий код демонстрирует другие возможности генераторов. Например, для сохранения и восстановления состояния генератора можно использовать операторы `<<` и `>>`.⁶ Следующие инструкции записывают состояние в строку с помощью строкового потока (см. раздел 15.10):

```
std::ostringstream engineState;
engineState << dre;
```

Это состояние представляет собой последовательность значений, разделенных пробелами. (Но она *не является* последовательностью случайных чисел!) Если изменить состояние генератора `dre` — например, генерируя числа, — а затем восстановить его, то генератор перейдет в состояние, которое было сохранено, и сгенерирует те же числа снова.

Следующий оператор пропускает два состояния:

```
dre.discard(2);
```

Таким образом, генератор `dre` будет находиться в таком состоянии, будто он уже сгенерировал два значения. Однако зачастую функция `discard()` при переборе состояний с помощью оператора `()` может работать быстрее.

17.1.3. Подробное описание генераторов

Как показано на рис. 17.1 (раздел 17.1), генераторы в стандартной библиотеке шаблонов разделяются на несколько категорий.

- **Основные генераторы**, реализующие основные алгоритмы генерации случайных значений.
 - Класс `std::linear_congruential_engine`
 - Класс `std::mersenne_twister_engine`
 - Класс `std::subtract_with_carry_engine`

⁶Вероятно, это первый и единственный интерфейс сериализации в стандартной библиотеке C++.

- **Адаптеры генераторов**, которые можно инициализировать (основными) генераторами.
 - Класс `std::discard_block_engine`, адаптирующий генератор, каждый раз отбрасывает заданное количество сгенерированных значений.
 - Класс `std::independent_bits_engine`, адаптирующий генератор для создания случайных значений с заданным количеством битов.
 - Класс `std::shuffle_order_engine`, адаптирующий генератор с помощью изменения порядка генерируемых им значений.
- **Адаптеры с предопределенными параметрами.**
 - `std::minstd_rand0`
 - `std::minstd_rand`
 - `std::mt19937`
 - `std::mt19937_64`
 - `std::ranlux24_base`
 - `std::ranlux48_base`
 - `std::ranlux24`
 - `std::ranlux48`
 - `std::knuth_b`

Например, тип `knuth_b` — это сокращение для выражения

```
shuffle_order_engine<linear_congruential_engine<uint_fast32_t,  
16807,  
0,  
2147483647>,  
256>
```

Кроме того, по умолчанию задается тип `std::default_random_engine`, зависящий от реализации. Это единственный тип генератора, не гарантирующий создания одной и той же последовательности на разных платформах.

В табл. 17.1 перечислены операции над генераторами случайных чисел. Все стандартные генераторы одного и того же типа изначально имеют одно и то же состояние, т.е. оператор `==` возвращает значение `true` и оба генератора создают одну и ту же последовательность случайных значений. Таким образом, для создания разных случайных значений необходимо задавать разные параметры *seed*.

Таблица 17.1. Операции над генераторами случайных чисел

Операция	Описание
<code>engine e</code>	Конструктор по умолчанию; создает генератор с начальным состоянием, заданным по умолчанию
<code>engine e (seed)</code>	Создает генератор, состояние которого соответствует параметру <i>seed</i>
<code>engine e (e2)</code>	Копирующий конструктор; копирует генератор (<i>e</i> и <i>e2</i> будут иметь одинаковые состояния)

Окончание табл. 17.1

Операция	Описание
<code>e.seed()</code>	Переводит генератор <code>e</code> в начальное состояние
<code>e.seed(seed)</code>	Переводит генератор <code>e</code> в состояние, соответствующее параметру <code>seed</code>
<code>e()</code>	Возвращает следующее случайное значение и переводит генератор в следующее состояние
<code>e.discard(n)</code>	Переводит генератор в n -е состояние (эквивалентно n вызовам <code>e()</code> , но может работать быстрее)
<code>e1 == e2</code>	Проверяет равенство состояний генераторов <code>e1</code> и <code>e2</code>
<code>e1 != e2</code>	Проверяет неравенство состояний генераторов <code>e1</code> и <code>e2</code>
<code>ostrm << e</code>	Записывает состояние генератора <code>e</code> в поток вывода <code>ostrm</code>
<code>istrm >> e</code>	Считывает новое состояние из потока ввода в генератор <code>e</code>

Состояние, считанное оператором `<<`, представляет собой список десятичных значений, разделенных пробелом. Несмотря на то что этот список может выглядеть как последовательность случайных значений, на самом деле *это не так*.

Стандарт гарантирует, что при чтении записанного состояния генератор того же типа перейдет в то же состояние (оператор `==` возвращает `true`, и генерируются одинаковые случайные значения).

17.1.4. Распределения

Как было указано выше, распределения преобразуют случайные значения, созданные генератором, в реальные и полезные случайные числа. Вероятность этих случайных чисел зависит от используемого распределения, параметры которого задаются программистом. В табл. 17.2 приведен обзор всех распределений, существующих в стандартной библиотеке C++.

Таблица 17.2. Распределения, предусмотренные стандартной библиотекой C++

Категория	Имя	Тип данных
Равномерные распределения	<code>uniform_int_distribution</code>	<i>IntType</i>
	<code>uniform_real_distribution</code>	<i>RealType</i>
Распределения Бернулли	<code>bernoulli_distribution</code>	<code>bool</code>
	<code>binomial_distribution</code>	<i>IntType</i>
	<code>geometric_distribution</code>	<i>IntType</i>
	<code>negative_binomial_distribution</code>	<i>IntType</i>
	<code>poisson_distribution</code>	<i>IntType</i>
Распределения Пуассона	<code>exponential_distribution</code>	<i>RealType</i>
	<code>gamma_distribution</code>	<i>RealType</i>
	<code>weibull_distribution</code>	<i>RealType</i>
	<code>extreme_value_distribution</code>	<i>RealType</i>

Окончание табл. 17.2

Категория	Имя	Тип данных
Нормальные распределения	<code>normal_distribution</code>	<i>RealType</i>
	<code>lognormal_distribution</code>	<i>RealType</i>
	<code>chi_squared_distribution</code>	<i>RealType</i>
	<code>cauchy_distribution</code>	<i>RealType</i>
	<code>fisher_f_distribution</code>	<i>RealType</i>
	<code>student_t_distribution</code>	<i>RealType</i>
Выборочные распределения	<code>discrete_distribution</code>	<i>IntType</i>
	<code>piecewise_constant_distribution</code>	<i>RealType</i>
	<code>piecewise_linear_distribution</code>	<i>RealType</i>

Почти все распределения представляют собой шаблонные классы, параметризованные типами генерируемых значений. Единственным исключением является класс `bernoulli_distribution`, представляющий собой обычный класс, потому что он генерирует только значения типа `bool`.

По умолчанию используются следующие типы:

- `int` — для *IntType*;
- `double` — для *RealType*;

Стандартная библиотека C++ содержит следующие специализации шаблонов:

- для *IntType* — `short`, `int`, `long`, `long long` и соответствующие им типы без зна-
ков;
- для *RealType* — `float`, `double`, `long double`.

В табл. 17.3 приведены операции над распределениями.

Таблица 17.3. Типы и операции над распределениями

Операция	Описание
<code>distr::result_type</code>	Арифметический тип генерируемых значений
<code>distr d</code>	Конструктор по умолчанию; создает распределение с параметрами, заданными по умолчанию
<code>distr d(args)</code>	Создает распределение, параметризованное списком аргументов <i>args</i>
<code>d(e)</code>	Возвращает следующее значение с помощью генератора <i>e</i> и переводит генератор <i>d</i> в состояние генератора <i>e</i>
<code>d.min()</code>	Возвращает минимальное значение
<code>d.max()</code>	Возвращает максимальное значение
<code>d1 == d2</code>	Проверяет равенство состояний генераторов <i>d1</i> и <i>d2</i>
<code>d1 != d2</code>	Проверяет неравенство состояний генераторов <i>d1</i> и <i>d2</i>
<code>ostrm << d</code>	Записывает состояние генератора <i>d</i> в поток вывода <i>ostrm</i>

Окончание табл. 17.3

Операция	Описание
<i>istrm</i> >> <i>d</i>	Считывает новое состояние из потока ввода <i>istrm</i> в генератор <i>d</i>
<i>distr</i> ::param_type	Тип параметров <i>distr</i>
<i>distr</i> <i>d</i> (<i>pt</i>)	Создает распределение с параметром param_type <i>pt</i>
<i>d</i> .param(<i>pt</i>)	Задаёт текущий параметр равным param_type <i>pt</i>
<i>d</i> .param()	Возвращает текущий параметр как значение типа param_type
<i>d</i> (<i>e</i> , <i>pt</i>)	Возвращает следующее значение с помощью генератора <i>e</i> и параметра param_type <i>pt</i> и переводит генератор в состояние <i>e</i>
<i>d</i> .param()	Возвращает значение параметра <i>param</i>

Распределение можно параметризовать, поэтому параметры *args* и *pt* зависят от конкретного распределения. Для параметризации можно

- передать список аргументов *args* конструктору;
- использовать член, соответствующий параметру, чтобы запросить его значение;
- использовать тип param_type, чтобы
 - передать эти параметры конструктору в виде одного аргумента;
 - передать эти параметры для генерации следующего значения;
 - запросить значения этих параметров.

Класс param_type должен иметь конструктор, которому можно передавать значения параметров, или именованные члены, возвращающие эти значения.

Например, равномерные распределения имеют два параметра, *a* и *b*, соответствующие минимуму и максимуму (см. раздел 17.1.5). В результате эти аргументы можно передавать по отдельности.

```
uniform_int_distribution<> d(0, 20); // инициализируем параметры "a" и "b"
d.a()                               // выясняем значение параметра "a"
d.b()                               // выясняем значение параметра "b"
d.param().a()                       // выясняем значение параметра "a"
d.param().b()                       // выясняем значение параметра "b"
```

Кроме того, можно просто передать объект класса param_type:

```
uniform_int_distribution<>::param_type pt(100, 200); // другая параметризация
d(e,pt)       // генерирует одно значение в соответствии с параметризацией pt
d.param(pt);  // позволяет всем сгенерированным значениям
               // использовать параметризацию pt
```

Выборочные распределения используют специальные конструкторы для передачи векторов значений или генератора значений.

Отметим, что максимальное значение, передаваемое распределениям как параметр, иногда может включаться в интервал, а иногда нет.

Для генераторов случайных чисел состояние, записанное с помощью оператора <<, представляет собой десятичные значения, разделенные пробелами. Несмотря на то что они могут выглядеть как список случайных чисел, *это не так*.

Следующая программа демонстрирует использование распределений:

```
// num/dist1.cpp

#include <random>
#include <map>
#include <string>
#include <iostream>
#include <iomanip>

template <typename Distr, typename Eng>
void distr (Distr d, Eng e, const std::string& name)
{
    // выводим min, max и четыре значения
    std::cout << name << ":" << std::endl;
    std::cout << "- min(): " << d.min() << std::endl;
    std::cout << "- max(): " << d.max() << std::endl;
    std::cout << "- values: " << d(e) << ' ' << d(e) << ' '
        << d(e) << ' ' << d(e) << std::endl;

    // выводим сгенерированные значения (преобразованные в целые числа)
    std::map<long long,int> valuecounter;
    for (int i=0; i<200000; ++i) {
        valuecounter[d(e)]++;
    }

    // и распечатываем полученное распределение
    std::cout << "====" << std::endl;
    for (auto elem : valuecounter) {
        std::cout << std::setw(3) << elem.first << ": "
            << elem.second << std::endl;
    }
    std::cout << "====" << std::endl;
    std::cout << std::endl;
}

int main()
{
    std::knuth_b e;

    std::uniform_real_distribution<> ud(0, 10);
    distr(ud,e,"uniform_real_distribution");

    std::normal_distribution<> nd;
    distr(nd,e,"normal_distribution");

    std::exponential_distribution<> ed;
    distr(ed,e,"exponential_distribution");

    std::gamma_distribution<> gd;
    distr(gd,e,"gamma_distribution");
}
```

На моем компьютере программа выдала следующий результат (представленный в двух столбцах):

<pre>uniform_real_distribution: - min(): 0 - max(): 10 - values: 8.30965 1.30427 9.47764 3.83416 ===== 0: 20087 1: 20057 2: 19878 3: 19877 4: 20005 5: 20118 6: 20063 7: 19886 8: 20003 9: 20026 ===== exponential_distribution: - min(): 0 - max(): 1.79769e+308 - values: 0.185167 2.03694 0.0536495 0.958636 ===== 0: 126487 1: 46436 2: 17120 3: 6294 4: 2326 5: 865 6: 283 7: 107 8: 52 9: 17 10: 6 11: 6 12: 1 =====</pre>	<pre>normal_distribution: - min(): 2.22507e-308 - max(): 1.79769e+308 - values: -0.131724 0.117963 -0.140331 0.538967 ===== -4: 9 -3: 245 -2: 4325 -1: 26843 0: 136947 1: 26987 2: 4377 3: 258 4: 9 ===== gamma_distribution: - min(): 0 - max(): 1.79769e+308 - values: 0.117964 1.60557 0.558526 1.21066 ===== 0: 126315 1: 46477 2: 17160 3: 6271 4: 2413 5: 866 6: 327 7: 109 8: 41 9: 12 10: 7 11: 1 12: 1 =====</pre>
--	---

17.1.5. Подробное описание распределений

Рассмотрим функции распределения, плотность вероятностей и их параметры.

uniform_int_distribution:

$$P(i|a,b) = \frac{1}{b-a+1}.$$

Параметры:

IntType **a** (по умолчанию: 0)

IntType **b** (по умолчанию: *limits::max()*)

uniform_real_distribution:

$$P(x|a,b) = \frac{1}{b-a}$$

Параметры:

RealType **a** (по умолчанию: 0.0)

RealType **b** (по умолчанию: 1.0)

bernoulli_distribution:

$$P(b|p) = \begin{cases} p, & \text{если } b = \text{true}, \\ 1-p, & \text{если } b = \text{false}. \end{cases}$$

Параметры:

double **p** (по умолчанию: 0.5)

binomial_distribution:

$$P(i|t, p) = \binom{t}{i} p^i (1-p)^{t-i}.$$

Параметры:

IntType **t** (по умолчанию: 1)

double **p** (по умолчанию: 0.5)

geometric_distribution:

$$P(i|p) = p(1-p)^i.$$

Параметры:

double **p** (по умолчанию: 0.5)

negative_binomial_distribution:

$$P(i|k, p) = \binom{k+i-1}{i} p^k (1-p)^i.$$

Параметры:

IntType **k** (по умолчанию: 1)

double **p** (по умолчанию: 0.5)

poisson_distribution:

$$P(i|\mu) = \frac{e^{-\mu} \mu^i}{i!}.$$

Параметры:

double **mean** (по умолчанию: 1.0)

exponential_distribution:

$$P(x|\lambda) = \lambda e^{-\lambda x}.$$

Параметры:

RealType **lambda** (по умолчанию: 1.0)

gamma_distribution:

$$P(x|\alpha, \beta) = \frac{e^{-x/\beta}}{\beta^\alpha \Gamma(\alpha)} x^{\alpha-1}.$$

Параметры:

RealType **alpha** (по умолчанию: 1.0)

RealType **beta** (по умолчанию: 1.0)

weibull_distribution:

$$P(x|a,b) = \frac{a}{b} \left(\frac{x}{b}\right)^{a-1} \exp\left(-\left(\frac{x}{b}\right)^a\right).$$

Параметры:

RealType a (по умолчанию: 1.0)*RealType b* (по умолчанию: 1.0)**extreme_value_distribution:**

$$P(x|a,b) = \frac{1}{b} \exp\left(\frac{a-x}{b} - \exp\left(\frac{a-x}{b}\right)\right).$$

Параметры:

RealType a (по умолчанию: 1.0)*RealType b* (по умолчанию: 1.0)**normal_distribution:**

$$P(x|\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\left(\frac{x-\mu}{2\sigma^2}\right)^2\right).$$

Параметры:

RealType mean (по умолчанию: 0.0)*RealType stddev* (по умолчанию: 1.0)**lognormal_distribution:**

$$P(x|m,s) = \frac{1}{sx\sqrt{2\pi}} \exp\left(-\frac{(\ln x - m)^2}{2s^2}\right).$$

Параметры:

RealType m (по умолчанию: 0.0)*RealType s* (по умолчанию: 1.0)**chi_squared_distribution:**

$$P(x|n) = \frac{x^{(n/2)-1} e^{-x/2}}{\Gamma(n/2) 2^{n/2}}.$$

Параметры:

RealType n (по умолчанию: 1)**cauchy_distribution:**

$$P(x|a,b) = \left(\pi b \left(1 + \left(\frac{x-a}{b} \right)^2 \right) \right)^{-1}.$$

Параметры:

RealType a (по умолчанию: 1.0)*RealType b* (по умолчанию: 1.0)

fisher_f_distribution:

$$P(x|m, n) = \frac{\Gamma((m+n)/2)}{\Gamma(m/2)\Gamma(n/2)} \left(\frac{m}{n}\right)^{m/2} x^{(m/2)-1} \left(1 + \frac{mx}{n}\right)^{-(m+n)/2}.$$

Параметры:

RealType **m** (по умолчанию: 1)

RealType **n** (по умолчанию: 1)

student_t_distribution:

$$p(x|n) = \frac{1}{\sqrt{n\pi}} \frac{\Gamma((n+1)/2)}{\Gamma(n/2)} \left(1 + \frac{x^2}{n}\right)^{-(n+1)/2}.$$

Параметры:

RealType **n** (по умолчанию: 1)

discrete_distribution:

$$P(i|p_0, \dots, p_{n-1}) = p_i.$$

Параметры:

vector<double> **probabilities**

Конструкторы получают списки инициализации, состоящие из чисел типа double, интервал чисел типа double и функциональный объект, создающий числа типа double.

piecewise_constant_distribution:

$$p(x|b_0, \dots, b_n, \rho_0, \dots, \rho_{n-1}) = \rho_i \text{ для } b_i \leq x < b_{i+1}.$$

Параметры:

vector<*RealType*> **intervals** (по умолчанию: не предусмотрено)

vector<*RealType*> **densities** (по умолчанию: не предусмотрено)

Конструктор получает интервал граничных значений и весов, список инициализации границ и генератор весов, а также число, минимум и максимум границ и генератор весов.

piecewise_linear_distribution:

$$p(x|b_0, \dots, b_n, \rho_0, \dots, \rho_{n-1}) = \rho_i \frac{b_{i+1} - x}{b_{i+1} - b_i} + \rho_{i+1} \frac{x - b_i}{b_{i+1} - b_i} \text{ для } b_i \leq x < b_{i+1}.$$

Параметры:

vector<*RealType*> **intervals**

vector<*RealType*> **densities**

Конструктор получает интервал граничных значений и весов, список инициализации границ и генератор весов, а также число, минимум и максимум границ и генератор весов.

Дальнейшие детали можно найти в спецификации стандартной библиотеки C++.

17.2. Комплексные числа

Стандартная библиотека C++ содержит шаблонный класс `complex<>`, предназначенный для работы с комплексными числами. Комплексные числа состоят из двух частей: действительной и мнимой, причем квадрат мнимой части является отрицательным числом. Иначе говоря, мнимая часть комплексного числа — это множитель перед квадратным корнем из -1 , который обозначается символом i .

Изменения, внесенные стандартом C++11

Стандарт C++98 определял практически все свойства класса `complex<>`. Они были определены уже при первой стандартизации языка C++. За исключением ключевого слова `constexpr` ни одно из новых функциональных свойств языка C++11 не оказало влияния на класс `complex<>` и его специализации. Было исправлено лишь несколько неточностей и добавлено небольшое количество усовершенствований.

- Добавлены операции, получающие комплексные числа как аргументы: `proj()`, `asin()`, `acos()`, `atan()`, `asin()`, `acosh()` и `atanh()`.
- Теперь с помощью функций `real(val)` и `imag(val)` действительную и мнимую части комплексного числа можно задать непосредственно.

17.2.1. Общее описание класса `complex<>`

Класс `complex<>` объявлен в заголовочном файле `<complex>`:

```
#include <complex>
```

В заголовочном файле `<complex>` класс `complex<>` определен следующим образом:

```
namespace std {
    template <typename T>
    class complex;
}
```

Шаблонный параметр `T` используется как скалярный тип и действительной, и мнимой части комплексного числа.

Кроме того, стандартная библиотека C++ содержит три специализации для типов `float`, `double` и `long double`:

```
namespace std {
    template<> class complex<float>;
    template<> class complex<double>;
    template<> class complex<long double>;
}
```

Эти типы позволяют выполнить определенную оптимизацию и безопасное преобразование одного типа комплексного числа в другой.

17.2.2. Примеры использования класса `complex<>`

Следующая программа демонстрирует некоторые из возможностей класса `complex<>` по созданию комплексных чисел, их выводу в разных представлениях и выполнению некоторых типичных операций над комплексными числами:

```
// num/complex1.cpp

#include <iostream>
#include <complex>
using namespace std;

int main()
{
    // комплексное числа с действительной и мнимой частями
    // - действительная часть: 4.0
    // - мнимая часть: 3.0
    complex<double> c1(4.0,3.0);

    // создаем комплексное число в полярных координатах
    // - модуль: 5.0
    // - аргумент: 0.75
    complex<float> c2(polar(5.0,0.75));

    // выводим комплексные числа с действительной и мнимой частями
    cout << "c1: " << c1 << endl;
    cout << "c2: " << c2 << endl;

    // выводим комплексные числа в полярных координатах
    cout << "c1: magnitude: " << abs(c1)
        << " (squared magnitude: " << norm(c1) << ") "
        << " phase angle: " << arg(c1) << endl;
    cout << "c2: magnitude: " << abs(c2)
        << " (squared magnitude: " << norm(c2) << ") "
        << " phase angle: " << arg(c2) << endl;

    // выводим сопряженные комплексные числа
    cout << "c1 conjugated: " << conj(c1) << endl;
    cout << "c2 conjugated: " << conj(c2) << endl;

    // выводим результат вычислений
    cout << "4.4 + c1 * 1.8: " << 4.4 + c1 * 1.8 << endl;

    // выводим сумму c1 и c2:
    // - примечание: типы разные
    cout << "c1 + c2: "
        << c1 + complex<double>(c2.real(),c2.imag()) << endl;

    // добавляем к c1 квадратный корень из c1 и выводим результат
    cout << "c1 += sqrt(c1): " << (c1 += sqrt(c1)) << endl;
}
```


Эта программа может вывести следующий результат (его точный вид зависит от конкретной реализации типа `double`):

```
c1: (4,3)
c2: (3.65844,3.40819)
c1: magnitude: 5 (squared magnitude: 25) phase angle: 0.643501
c2: magnitude: 5 (squared magnitude: 25) phase angle: 0.75
c1 conjugated: (4,-3)
c2 conjugated: (3.65844,-3.40819)
4.4 + c1 * 1.8: (11.6,5.4)
c1 + c2: (7.65844,6.40819)
c1 += sqrt(c1): (6.12132,3.70711)
```

Второй пример содержит цикл, в котором вводятся два комплексных числа и первое число возводится в степень второго:

```
// num/complex2.cpp

#include <iostream>
#include <complex>
#include <cstdlib>
#include <limits>

#include <cstdio>

using namespace std;
int main()
{
    complex<long double> c1, c2;
    while (cin.peek() != EOF) {

        // вводим первое комплексное число
        cout << "complex number c1: ";
        cin >> c1;
        if (!cin) {
            cerr << "input error" << endl;
            return EXIT_FAILURE;
        }

        // вводим второе комплексное число
        cout << "complex number c2: ";
        cin >> c2;

        if (!cin) {
            cerr << "input error" << endl;
            return EXIT_FAILURE;
        }

        if (c1 == c2) {
            cout << "c1 and c2 are equal !" << endl;
        }

        cout << "c1 raised to the c2: " << pow(c1,c2)
```

```
        << endl << endl;

        // пропускаем остальную часть строки
        cin.ignore(numeric_limits<int>::max(), '\\n');
    }
}
```

В табл. 17.4 приведены возможные ввод и вывод программы. Отметим, что комплексное число можно задавать в виде отдельной действительной части в скобках или без них, а также в виде пары чисел, представляющих собой действительную и мнимую части, заключенные в скобки и разделенные запятыми.

Таблица 17.4. Возможный ввод и вывод программы complex2.cpp

c1	c2	Вывод
2	2	c1 raised to c2: (4,0)
(16)	0.5	c1 raised to c2: (4,0)
(8,0)	0.333333333	c1 raised to c2: (2,0)
0.99	(5)	c1 raised to c2: (0.95099,0)
(0,2)	2	c1 raised to c2: (-4,4.89843e-16)
(1.7,0.3)	0	c1 raised to c2: (1,0)
(3,4)	(-4,3)	c1 raised to c2: (4.32424e-05,8.91396e-05)
(1.7,0.3)	(4.3,2.8)	c1 raised to c2: (-4.17622,4.86871)

17.2.3. Операции над комплексными числами

Шаблонный класс `complex<>` содержит операции, описанные в следующих подразделах.

Создание, копирование и присваивание

В табл. 17.5 приведены конструкторы и операции присваивания для класса `complex`. Конструкторы могут получать начальные значения действительной и мнимой частей. Если эти части не заданы, они инициализируются конструктором по умолчанию в зависимости от типа их значений.

Таблица 17.5. Конструкторы и операции присваивания для класса complex<>

Выражение	Описание
<code>complex c</code>	Создает комплексное число с нулевыми действительной и мнимой частями, т.е. $0 + 0i$
<code>complex c(1.3)</code>	Создает комплексное число, у которого действительная часть равна 1.3, а мнимая — 0, т.е. $1.3 + 0i$
<code>complex c(1.3,4.2)</code>	Создает комплексное число, у которого действительная часть равна 1.3, а мнимая — 4.2, т.е. $1.3 + 4.2i$
<code>complex c1(c2)</code>	Создает комплексное число <code>c1</code> как копию <code>c2</code>

Окончание табл. 17.5

Выражение	Описание
<code>polar(4.2)</code>	Создает временное комплексное число в полярных координатах (модуль ρ равен 4.2, а аргумент θ — 0)
<code>polar(4.2, 0.75)</code>	Создает временное комплексное число в полярных координатах (модуль ρ равен 4.2, а аргумент θ — 0,75)
<code>conj(c)</code>	Создает временное комплексное число, которое является сопряженным к комплексному числу c (комплексное число с противоположной по знаку мнимой частью)
<code>proj(c)</code>	Создает временное комплексное число по проекции числа c на сферу Римана (начиная со стандарта C++11)
<code>c1 = c2</code>	Присваивает число $c2$ числу $c1$
<code>c1 += c2</code>	Добавляет число $c2$ к числу $c1$
<code>c1 -= c2</code>	Вычитает число $c2$ из числа $c1$
<code>c1 *= c2</code>	Умножает число $c2$ на число $c1$ и присваивает результат числу $c1$
<code>c1 /= c2</code>	Делит число $c1$ на число $c2$ и присваивает результат числу $c1$

Значение существующего комплексного числа можно изменить только с помощью операторов присваивания. Сокращенные операторы присваивания `+=`, `-=`, `*=` и `/=` суммируют, вычитают, умножают и делят два комплексных числа, записывая результат в первый операнд.

Вспомогательная функция `polar()` позволяет создать комплексное число, инициализированное полярными координатами (модуль и аргумент, заданный в радианах):

```
// создаем комплексное число, инициализированное полярными координатами
std::complex<double> c2(std::polar(4.2, 0.75));
```

При неявном преобразовании комплексных чисел во время их создания возникает одна проблема. Например, следующее выражение является корректным:

```
std::complex<float> c2(std::polar(4.2, 0.75)); // ОК
```

Однако конструкция, приведенная ниже, является ошибочной.

```
std::complex<float> c2 = std::polar(4.2, 0.75); // ОШИБКА
```

Эта проблема рассматривается в следующем подразделе.

Вспомогательная функция `conj()` позволяет создать комплексное число, инициализированное значением, сопряженным с другим комплексным числом (т. е. комплексным числом с противоположным знаком мнимой части).

```
std::complex<double> c1(1.1, 5.5);
std::complex<double> c2(conj(c1)); // инициализируем число c2 комплексным числом
// complex<double>(1.1, -5.5)
```

В стандарте C++11 введена вспомогательная функция `proj()`, создающая комплексное число, представляющее собой проекцию другого комплексного числа на сферу Римана.

Неявные преобразования типов

Конструкторы для специализаций `float`, `double` и `long double` разработаны так, чтобы безопасные преобразования, например `complex<float>` в `complex<double>`, можно было выполнять неявно, а менее безопасные, такие как `complex<long double>` в `complex<double>`, только явно.

```
std::complex<float> cf;
std::complex<double> cd;
std::complex<long double> cld;
...
std::complex<double> cd1 = cf; // ОК: безопасное преобразование
std::complex<double> cd2 = cld; // ОШИБКА: не неявное преобразование
std::complex<double> cd3(cld); // ОК: явное преобразование
```

Кроме того, не существует конструкторов, создающих комплексное число из комплексного числа другого типа. В частности, нельзя преобразовать тип `complex` с целочисленными действительной и мнимой частями в тип `complex` с действительной и мнимой частями типа `float`, `double` или `long double`. Однако эти преобразования можно выполнять при передаче вещественной и мнимой частей в виде отдельных аргументов.

```
std::complex<double> cd;
std::complex<int> ci;
...
std::complex<double> cd4 = ci; // ОШИБКА: неявное преобразование не допускается
std::complex<double> cd5(ci); // ОШИБКА: явное преобразование не допускается
std::complex<double> cd6(ci.real(), ci.imag()); // ОК
```

К сожалению, операции присваивания позволяют выполнять менее безопасные преобразования и определены в виде шаблонных функций для всех типов. Следовательно, можно присваивать комплексные числа любых типов, если типы значений действительной и мнимой частей допускают преобразования⁷.

```
std::complex<double> cd;
std::complex<long double> cld;
```

```
std::complex<int> ci;
...
cd = ci; // ОК
cd = cld; // ОК
```

Эта проблема относится также к функциям `polar()`, `conj()` и `proj()`. Например, следующая конструкция работает отлично:

```
std::complex<float> c2(std::polar(4.2, 0.75)); // ОК
```

Однако конструкция с операцией присваивания является ошибочной:

```
std::complex<float> c2 = std::polar(4.2, 0.75); // ОШИБКА
```

⁷ То, что конструкторы специализаций комплексных чисел допускают только безопасные неявные преобразования, а операции присваивания допускают любые неявные преобразования, вероятно, является ошибкой разработчиков стандарта.

Причина заключается в том, что выражение

```
std::polar(4.2, 0.75)
```

создает временный объект класса `complex<double>`, а неявное преобразование типа `complex<double>` в тип `complex<float>` не определено⁸.

Доступ к значениям

В табл. 17.6 перечислены функции, обеспечивающие доступ к атрибутам комплексных чисел.

Таблица 17.6. Операции доступа к членам класса `complex<>`

Выражение	Описание
<code>real(c)</code>	Возвращает значение действительной части (как глобальная функция)
<code>c.real()</code>	Возвращает значение действительной части (как функция-член класса)
<code>c.real(1.7)</code>	Присваивает 1.7 как новую действительную часть (начиная со стандарта C++11)
<code>imag(c)</code>	Возвращает значение мнимой части (как глобальная функция)
<code>c.imag()</code>	Возвращает значение мнимой части (как функция-член класса)
<code>c.imag(1.7)</code>	Присваивает 1.7 как новую мнимую часть (начиная со стандарта C++11)
<code>abs(c)</code>	Возвращает абсолютное значение числа c ($\sqrt{c.\text{real}()^2 + c.\text{imag}()^2}$)
<code>norm(c)</code>	Возвращает абсолютное значения числа c , возведенное в квадрат ($c.\text{real}()^2 + c.\text{imag}()^2$)
<code>arg(c)</code>	Возвращает аргумент j представления комплексного числа c в полярных координатах (эквивалент <code>atan2(c.imag(), c.real())</code>)

До принятия стандарта C++11 функции `real()` и `imag()` обеспечивали только доступ к действительной и мнимой частям комплексного числа. Изменить только действительную или только мнимую части комплексного числа можно было, только присвоив ему новое комплексное число. Например, следующий оператор задает мнимую часть числа c равной 3.7:

```
std::complex<double> c;
...
c = std::complex<double>(c.real(), 3.7); // начиная со стандарта C++11: c.imag(3.7)
```

Операции сравнения

Для сравнения комплексных чисел предусмотрены только проверки равенства и неравенства (табл. 17.7). Операторы `==` и `!=` определены как глобальные функции, поэтому один из операндов может быть скалярным значением. Операнд, заданный скалярным

⁸ В общем случае синтаксис инициализации с оператором `=` требует, чтобы было возможным неявное преобразование типов.

значением, интерпретируется как действительная часть, а мнимая часть по умолчанию равна нулю, определенным для данного типа (обычно 0).

Таблица 17.7. Операции сравнения в классе `complex<>`

Выражение	Описание
<code>c1 == c2</code>	Проверяет равенство <code>c1</code> и <code>c2</code> (<code>c1.real()==c2.real() && c1.imag()==c2.imag()</code>)
<code>c == 1.7</code>	Проверяет равенство <code>c</code> и <code>1.7</code> (<code>c.real()==1.7 && c.imag()==0.0</code>)
<code>1.7 == c</code>	Проверяет равенство <code>c</code> и <code>1.7</code> (<code>c.real()==1.7 && c.imag()==0.0</code>)
<code>c1 != c2</code>	Проверяет неравенство <code>c1</code> и <code>c2</code> (<code>c1.real()!=c2.real() c1.imag()!=c2.imag()</code>)
<code>c != 1.7</code>	Проверяет неравенство <code>c</code> и <code>1.7</code> (<code>c.real()!=1.7 c.imag()!=0.0</code>)
<code>1.7 != c</code>	Проверяет неравенство <code>c</code> и <code>1.7</code> (<code>c.real()!=1.7 c.imag()!=0.0</code>)

Другие операторы сравнения, например `<`, не определены. Несмотря на то что для комплексных чисел теоретически можно определить отношение “больше”, такое упорядочение оказывается искусственным и практически бесполезным. Например, бессмысленно сравнивать комплексные числа по модулю, потому что два совершенно разных комплексных числа (например, `1` и `-1`) могут иметь одинаковые модули. Впрочем, можно придумать вполне корректное отношение “больше”. Например, можно считать, что комплексное число `c1` меньше комплексного числа `c2`, если $|c1| < |c2|$, а при равенстве модулей, если $\arg(c1) < \arg(c2)$. Однако такой критерий практически не имеет математического смысла⁹.

В результате тип `complex<>` невозможно использовать в качестве типа элемента ассоциативного контейнера, если не задать пользовательский критерий сортировки. Причина заключается в том, что ассоциативные контейнеры используют функциональный объект `less<>`, который вызывает оператор `<` для сортировки элементов (см. раздел 6.11.1).

Реализовав пользовательский оператор `<`, можно сортировать комплексные числа и хранить их в ассоциативных контейнерах. При этом следует быть очень осторожным и не засорять стандартное пространство имен.

Рассмотрим пример:

```
template <typename T>
bool operator< (const std::complex<T>& c1,
               const std::complex<T>& c2)
{
    return std::abs(c1)<std::abs(c2) ||
           (std::abs(c1)==std::abs(c2) &&
            std::arg(c1)<std::arg(c2));
}
```

Арифметические операции

Для комплексных чисел определены четыре основные арифметические операции, а также операции применения знаков “плюс” и “минус” (табл. 17.8).

⁹ Благодарю за это замечание Дэвида Вандевурда (David Vandevoorde).

Таблица 17.8. Арифметические операции над классом `complex<>`

Выражение	Описание
<code>c1 + c2</code>	Возвращает сумму <code>c1</code> и <code>c2</code>
<code>c + 1.7</code>	Возвращает сумму <code>c</code> и <code>1.7</code>
<code>1.7 + c</code>	Возвращает сумму <code>1.7</code> и <code>c</code>
<code>c1 - c2</code>	Возвращает разность <code>c1</code> и <code>c2</code>
<code>c - 1.7</code>	Возвращает разность <code>c</code> и <code>1.7</code>
<code>1.7 - c</code>	Возвращает разность <code>1.7</code> и <code>c</code>
<code>c1 * c2</code>	Возвращает произведение <code>c1</code> и <code>c2</code>
<code>c * 1.7</code>	Возвращает произведение <code>c</code> и <code>1.7</code>
<code>1.7 * c</code>	Возвращает произведение <code>1.7</code> и <code>c</code>
<code>c1 / c2</code>	Возвращает частное от деления <code>c1</code> на <code>c2</code>
<code>c / 1.7</code>	Возвращает частное от деления <code>c</code> на <code>1.7</code>
<code>1.7 / c</code>	Возвращает частное от деления <code>1.7</code> на <code>c</code>
<code>- c</code>	Возвращает число <code>c</code> с противоположным знаком
<code>+ c</code>	Возвращает число <code>c</code>
<code>c1 += c2</code>	Эквивалент <code>c1 = c1 + c2</code>
<code>c1 -= c2</code>	Эквивалент <code>c1 = c1 - c2</code>
<code>c1 *= c2</code>	Эквивалент <code>c1 = c1 * c2</code>
<code>c1 /= c2</code>	Эквивалент <code>c1 = c1 / c2</code>

Операции ввода-вывода

Класс `complex` поддерживает общие операторы ввода-вывода `<<` и `>>` (табл. 17.9).

Оператор вывода записывает комплексное число в соответствии с текущим состоянием потока, используя формат:

(realpart, imagpart)

Таблица 17.9. Операции ввода-вывода для класса `complex<>`

Выражение	Описание
<code>strm << c</code>	Записывает комплексное число <code>c</code> в поток вывода <code>strm</code>
<code>strm >> c</code>	Считывает комплексное число <code>c</code> из потока ввода <code>strm</code>

В частности, операция вывода является эквивалентом следующей реализации:

```
template <typename T, typename charT, typename traits>
std::basic_ostream<charT,traits>&
operator << (std::basic_ostream<charT,traits>& strm,
            const std::complex<T>& c)
{
    // временная строка значений для вывода с одним аргументом
```

```
std::basic_ostringstream<charT,traits> s;
s.flags(strm.flags());           // копируем флаги потока
s.imbue(strm.getloc());          // копируем локальный контекст потока
s.precision(strm.precision());  // копируем точность потока

// подготавливаем строку значений
s << '(' << c.real() << ',' << c.imag() << ')';

// записываем строку значений
strm << s.str();
return strm;
}
```

Операция ввода дает возможность читать комплексное число в одном из следующих форматов:

```
(realpart, imagpart)
(realpart)
realpart
```

Если ни один из следующих символов не соответствует ни одному из форматов, то устанавливается флаг `ios::failbit`, что может привести к генерированию соответствующего исключения (см. раздел 15.4.4).

К сожалению, задать разделитель между действительной и мнимой частями комплексного числа невозможно. По этой причине, если вы используете в качестве “десятичной точки” запятую (как в Германии), то ввод-вывод будет выглядеть очень странно. Например, комплексное число с действительной частью 4.6 и мнимой частью 2.7 записывается в виде (4,6,2,7).

Пример использования операций ввода-вывода приведен в разделе 17.2.2.

Трансцендентные функции

В табл. 17.10 перечислены трансцендентные функции (тригонометрические, экспоненциальные и др.) для класса `complex`.

Таблица 17.10. Трансцендентные функции для класса `complex`

Выражение	Описание
<code>pow(c, 3)</code>	Возведение комплексного числа в степень: c^3
<code>pow(c, 1.7)</code>	Возведение комплексного числа в степень: $c^{1.7}$
<code>pow(c1, c2)</code>	Возведение комплексного числа в степень: $c_1^{c_2}$
<code>pow(1.7, c)</code>	Возведение комплексного числа в степень: 1.7^c
<code>exp(c)</code>	Возведение в степень c по основанию e : e^c
<code>sqrt(c)</code>	Квадратный корень из c : $()$
<code>log(c)</code>	Комплексный натуральный логарифм c по основанию e ($\ln c$)
<code>log10(c)</code>	Комплексный десятичный логарифм c ($\lg c$)
<code>sin(c)</code>	Синус c ($\sin c$)

Окончание табл. 7.10

Выражение	Описание
<code>cos(c)</code>	Косинус c (<code>cos c</code>)
<code>tan(c)</code>	Тангенс c (<code>tan c</code>)
<code>sinh(c)</code>	Гиперболический синус c (<code>sinh c</code>)
<code>cosh(c)</code>	Гиперболический косинус c (<code>cosh c</code>)
<code>tanh(c)</code>	Гиперболический тангенс c (<code>tanh c</code>)
<code>asin(c)</code>	Арксинус c (начиная со стандарта C++11)
<code>acos(c)</code>	Аркосинус c (начиная со стандарта C++11)
<code>atan(c)</code>	Арктангенс c (начиная со стандарта C++11)
<code>asinh(c)</code>	Гиперболический арксинус c (начиная со стандарта C++11)
<code>acosh(c)</code>	Гиперболический аркосинус c (начиная со стандарта C++11)
<code>atanh(c)</code>	Гиперболический арктангенс c (начиная со стандарта C++11)

17.2.4. Подробное описание класса `complex<>`

В этом подразделе подробно описываются все операции класса `complex<>`. В следующих определениях символ T обозначает шаблонный параметр класса `complex<>`, являющийся типом действительной и мнимой частей значения класса `complex`.

Определения типов

`complex::value_type`

- Тип действительной и мнимой частей.

Создание, копирование и присваивание

`complex::complex()`

- Конструктор по умолчанию.
- Создает комплексное число, в котором действительная и мнимая части инициализируются с помощью явного вызова их конструктора по умолчанию. Таким образом, для элементарных типов начальное значение действительной и мнимой частей равно 0 (подробно о значениях, заданных по умолчанию для элементарных типов, см. в разделе 3.2.1).

`complex::complex(const T& re)`

- Создает комплексное число, в котором аргумент re является значением действительной части, а мнимая часть инициализируется с помощью явного вызова ее конструктора по умолчанию (0 для элементарных типов).
- Этот конструктор также определяет автоматическое преобразования T в `complex`.

complex::complex (const T& *re*, const T& *im*)

- Создает комплексное число, в котором аргумент *re* является действительной частью, а *im* — мнимой.

complex polar (const T& *rho*)

complex polar (const T& *rho*, const T& *theta*)

- Обе версии создают и возвращают комплексное число, инициализированное полярными координатами.
- Аргумент *rho* задает модуль.
- Аргумент *theta* задает аргумент в радианах (по умолчанию: 0).

complex conj (const complex& *cmplx*)

- Создает и возвращает комплексное число, инициализированное комплексным числом, сопряженным с *cmplx* (мнимая часть имеет противоположный знак).

complex proj (const complex& *cmplx*)

- Создает и возвращает временное комплексное число на основе проекции x на сферу Римана.
- Эквивалент функции `cproj()` из языка C.
- Доступна по стандарту C++11.

complex::complex (const complex& *cmplx*)

- Копирующий конструктор.
- Создает новое комплексное число в виде копии аргумента *cmplx*.
- Копирует действительную и мнимую части.
- Эта функция существует как в шаблонном, так и в не шаблонном варианте (см. раздел 3.2). Таким образом, она поддерживает автоматическое преобразование элементарных типов.
- Однако конструктор по умолчанию ограничен специализациями для типов `float`, `double` и `long double`, поэтому менее безопасные преобразования — из `double` и `long double` в `float`, а также из `long double` в `double` — в неявном виде невозможны. Более подробно этот вопрос рассмотрен в разделе 17.2.3.

complex& complex::operator = (const complex& *cmplx*)

- Присваивает комплексное число *cmplx*.
- Возвращает `*this`.
- Эта функция существует как в шаблонном, так и в не шаблонном варианте (см. раздел 3.2). Таким образом, она поддерживает автоматическое преобразование элементарных типов. (Это относится и к специализациям, предусмотренным стандартной библиотекой C++.)

```

complex& complex::operator += (const complex& cmplx)
complex& complex::operator -= (const complex& cmplx)
complex& complex::operator *= (const complex& cmplx)
complex& complex::operator /= (const complex& cmplx)

```

- Эти операции вычисляют сумму, разность, произведение и частное от деления комплексных чисел *cmplx* и **this* соответственно, записывая результат в **this*.
- Возвращает **this*.
- Эти функции существуют как в шаблонном, так и в не шаблонном варианте (см. раздел 3.2). Таким образом, она поддерживает автоматическое преобразование элементарных типов. (Это относится и к специализациям, предусмотренным стандартной библиотекой C++.)

Отметим, что операции присваивания — это единственные функции, позволяющие модифицировать значение существующего объекта класса *complex*.

Доступ к элементам

```

T complex::real () const
T real (const complex& cmplx)
T complex::imag () const
T imag (const complex& cmplx)

```

- Эти функции возвращают действительную или мнимую часть соответственно.
- Отметим, что возвращаемое значение не является ссылкой. Таким образом, эти функции нельзя использовать для модификации действительной и мнимой частей. Для изменения только действительной или только мнимой части необходимо присвоить новое комплексное число (см. раздел 17.2.3).

```

T complex::real (const T& re)
T complex::imag (const T& im)

```

- Эти функции присваивают аргументы *re* или *im* как новую действительную или мнимую часть соответственно.
- Доступны начиная со стандарта C++11. До принятия стандарта C++11 для модификации только действительной или только мнимой части приходилось присваивать новое комплексное число (см. раздел 17.2.3).

```

T abs (const complex& cmplx)

```

- Возвращает абсолютное значение (модуль) аргумента *cmplx*.
- Абсолютное значение равно $\sqrt{cmplx.real()^2 + cmplx.imag()^2}$.

```

T norm (const complex& cmplx)

```

- Возвращает модуль числа *cmplx* в квадрате.
- Квадрат абсолютного значения равен $cmplx.real()^2 + cmplx.imag()^2$.

T **arg** (const *complex& cmplx*)

- Возвращает аргумент (j) числа *cmplx* в полярной системе координат.
- Эквивалент функции `atan2 (cmplx.imag(), cmplx.real())`.

Операции ввода-вывода

ostream& operator << (*ostream& strm*, const *complex& cmplx*)

- Записывает значение *cmplx* в поток *strm*, используя формат (*realpart*, *imagpart*)
- Возвращает *strm*.
- Точное описание этой операции см. в разделе 17.2.3.

istream& operator >> (*istream& strm*, *complex& cmplx*)

- Считывает новое значение из потока *strm* в *cmplx*.
- Корректными являются форматы

(*realpart*, *imagpart*)

(*realpart*)

realpart

- Возвращение *strm*.
- Точное описание этой операции см. в разделе 17.2.3.

Операции

complex operator + (const *complex& cmplx*)

- Положительный знак.
- Возвращает *cmplx*.

complex operator - (const *complex& cmplx*)

- Отрицательный знак.
- Возвращает значение *cmplx* с отрицательными действительной и мнимой частями.

complex binary-op (const *complex& cmplx1*, const *complex& cmplx2*)

complex binary-op (const *complex& cmplx*, const T& *value*)

complex binary-op (const T& *value*, const *complex& cmplx*)

- Все версии возвращают комплексное число, являющееся результатом операции **binary-op**.
- Операция **binary-op** может быть одной из следующих:

`operator +`

`operator -`

`operator *`

`operator /`

- Если в качестве типа элемента задается скалярное значение, оно интерпретируется как действительная часть, а мнимая часть имеет значение, заданное по умолчанию (0 для элементарных типов).

```
bool comparison (const complex& cmplx1, const complex& cmplx2)
```

```
bool comparison (const complex& cmplx, const T& value)
```

```
bool comparison (const T& value, const complex& cmplx)
```

- Возвращает результат сравнения двух комплексных чисел или результат сравнения комплексного числа со скалярным значением.
- Операция **comparison** может быть одной из двух:
operator ==
operator !=
- Если в качестве типа элемента задается скалярное значение, оно интерпретируется как действительная часть, а мнимая часть имеет значение, заданное по умолчанию (0 для элементарных типов).
- Операторы <, <=, > и >= не предусмотрены.

Трансцендентные функции

```
complex pow (const complex& base, const T& exp)
```

```
complex pow (const complex& base, const complex& exp)
```

```
complex pow (const T& base, const complex& exp)
```

- Все версии возвращают комплексное число *base* в степени *exp*:
 $\exp(\exp \cdot \log(\textit{base}))$
- Точки ветвления лежат на отрицательном луче действительной прямой.
- Значение, возвращаемое функцией **pow**(0, 0), зависит от реализации.

```
complex exp (const complex& cmplx)
```

- Возвращает комплексное число *e* в степени *cmplx*.

```
complex sqrt (const complex& cmplx)
```

- Возвращает квадратный корень комплексного числа *cmplx*, лежащий в правой полуплоскости.
- Если аргумент является отрицательным действительным числом, то значение лежит на положительном луче мнимой оси.
- Точки ветвления лежат на отрицательном луче действительной прямой.

```
complex log (const complex& cmplx)
```

- Возвращает натуральный логарифм числа *cmplx*.
- Если *cmplx* является отрицательным действительным числом, то $\text{imag}(\log(\textit{cmplx}))$ равно π .
- Точки ветвления лежат на отрицательном луче действительной прямой.

complex log10 (const *complex& cmplx*)

- Возвращает десятичный логарифм числа *cmplx*.
- Эквивалент $\log(cmplx) / \log(10)$.
- Точки ветвления лежат на отрицательном луче действительной прямой.

complex sin (const *complex& cmplx*)

complex cos (const *complex& cmplx*)

complex tan (const *complex& cmplx*)

complex sinh (const *complex& cmplx*)

complex cosh (const *complex& cmplx*)

complex tanh (const *complex& cmplx*)

complex asin (const *complex& cmplx*)

complex acos (const *complex& cmplx*)

complex atan (const *complex& cmplx*)

complex asinh (const *complex& cmplx*)

complex acosh (const *complex& cmplx*)

complex atanh (const *complex& cmplx*)

- Эти операции возвращают значение соответствующей тригонометрической функции от аргумента *cmplx*.
- Обратные тригонометрические операции (имя которых начинается с буквы **a**) доступны начиная со стандарта C++11.

17.3. Глобальные числовые функции

Заголовочные файлы `<cmath>` и `<cstdlib>` содержат определения функций, унаследованных от языка C (табл. 17.11 и 17.12)¹⁰.

Таблица 17.11. Функции заголовочного файла `<cmath>`

Функция	Описание
<code>pow()</code>	Возведение в степень
<code>exp()</code>	Экспонента
<code>sqrt()</code>	Квадратный корень
<code>log()</code>	Натуральный логарифм
<code>log10()</code>	Десятичный логарифм
<code>sin()</code>	Синус
<code>cos()</code>	Косинус
<code>tan()</code>	Тангенс
<code>sinh()</code>	Гиперболический синус

¹⁰ По историческим причинам некоторые числовые функции определены в заголовочном файле `<cstdlib>`, а не в `<cmath>`.

Окончание табл. 17.11

Функция	Описание
<code>cosh()</code>	Гиперболический косинус
<code>tanh()</code>	Гиперболический тангенс
<code>asin()</code>	Арксинус
<code>acos()</code>	Аркосинус
<code>atan()</code>	Арктангенс
<code>atan2()</code>	Арктангенс частного
<code>asinh()</code>	Гиперболический арксинус (начиная со стандарта C++11)
<code>acosh()</code>	Гиперболический косинус (начиная со стандарта C++11)
<code>atanh()</code>	Гиперболический арктангенс (начиная со стандарта C++11)
<code>ceil()</code>	Число с плавающей точкой, округленное для следующего целого числа
<code>floor()</code>	Число с плавающей точкой, округленное для предыдущего целого числа
<code>fabs()</code>	Абсолютное значение числа с плавающей точкой
<code>fmod()</code>	Остаток от деления чисел с плавающей точкой (по модулю)
<code>frexp()</code>	Преобразует число с плавающей точкой в целую и дробную части
<code>ldexp()</code>	Умножает число с плавающей точкой на степень двойки
<code>modf()</code>	Выделяет из числа с плавающей точкой целую и дробную части со знаками

В отличие от языка C, в языке C++ некоторые операции перегружены для разных типов. В итоге некоторые числовые функции из языка C оказались устаревшими. Например, в языке C есть функции `abs()`, `labs()`, `llabs()`, `fabs()`, `fabsf()` и `fabsl()` для обработки абсолютных значений типа `int`, `long`, `long long`, `double`, `float()` и `long double` соответственно. В языке C++ функция `abs()` является перегруженной, поэтому ее можно применять ко всем указанным выше типам.

Таблица 17.12. Числовые функции из заголовочного файла `<cstdlib>`

Функция	Описание
<code>abs()</code>	Абсолютное значение типа <code>int</code>
<code>labs()</code>	Абсолютное значение типа <code>long</code>
<code>llabs()</code>	Абсолютное значение типа <code>long long</code> (начиная со стандарта C++11)
<code>div()</code>	Частное и остаток от деления чисел типа <code>int</code>
<code>ldiv()</code>	Частное и остаток от деления чисел типа <code>long</code>
<code>lldiv()</code>	Частное и остаток от деления чисел типа <code>long long</code> (начиная со стандарта C++11)
<code>srand()</code>	Генератор случайных чисел (создает начальное значение для новой последовательности)
<code>rand()</code>	Генератор случайных чисел (создает следующее число в последовательности)

В частности, все числовые функции для целочисленных значений перегружены для типов `int`, `long` и `long long`, а все числовые функции для чисел с плавающей точкой перегружены для типов `float`, `double` и `long double`.

Однако у перегрузки есть важный побочный эффект: если передать целочисленное значение функции, для которой существует только версия, перегруженная для чисел с плавающей точкой, то выражение станет неоднозначным¹¹:

```
std::sqrt(7) // НЕОДНОЗНАЧНОСТЬ: sqrt(float), sqrt(double) или
              // sqrt(long double)?
```

Вместо этого следует писать

```
std::sqrt(7.0) // ОК
```

или, если используется переменная,

```
int x;
...
std::sqrt(float(x)) // ОК
```

Разработчики библиотек решают эту проблему по-разному: одни используют перегрузку, другие поддерживают стандартное поведение (перегрузку для всех типов с плавающей точкой), третьи реализуют перегрузку для всех числовых типов вообще, а некоторые позволяют выбирать стратегию в зависимости от препроцессора. Таким образом, на практике неоднозначность может возникнуть, а может и не возникнуть. Для того чтобы написать переносимый код, всегда следует писать программу так, чтобы аргументы точно соответствовали ожидаемым типам.

17.4. Массивы значений

Еще со времен стандарта C++98 стандартная библиотека C++ содержит класс `valarray`, предназначенный для обработки массивов числовых значений.

Предназначение массивов значений

Массив значений реализует математическую концепцию линейной последовательности значений. Он имеет одно измерение, но с помощью специальных приемов вычисления индексов и мощных средств выделения подмножеств создает иллюзию большей размерности. Следовательно, массив значений можно использовать как для векторных, так и для матричных вычислений, а также для эффективной обработки систем полиномиальных уравнений.

С формальной точки зрения массивы значений — это одномерные массивы элементов, последовательно пронумерованных начиная с нуля. Массивы значений позволяют выполнять операции над всеми элементами или подмножествами одного или нескольких массивов значений. Например, можно вычислить выражение

$$z = a * x * x + b * x + c$$

в котором объекты `a`, `b`, `c`, `x` и `z` могут быть массивами, содержащими сотни числовых значений. Как видим, числовые массивы упрощают обозначения.

¹¹ Благодарю Дэвида Вандевурда за это замечание.

Кроме того, массивы значений обеспечивают высокую производительность обработки данных, поскольку классы обеспечивают оптимизацию, предотвращая создание временных объектов во время выполнения оператора. Эта оптимизация возможна благодаря тому, что у массивов значений нет псевдонимов. Иначе говоря, доступ к любому значению неконстантного массива значений осуществляется по уникальному пути. Это позволяет лучше оптимизировать операции с массивами, поскольку компилятору не приходится учитывать возможность доступа к данным по другому пути.

К тому же специальные интерфейсы и вспомогательные классы позволяют ограничивать работу подмножеством элементов массива значений и работать с многомерными массивами. Благодаря этому массивы значений позволяют реализовывать векторные и матричные вычисления и создавать соответствующие классы.

Проблемы, связанные с массивами значений

Классы массивов значений спроектированы не очень хорошо. Фактически никто и не пытался проверить работоспособность итоговой спецификации, потому что никто не считал себя ответственным за разработку этих классов. Люди, включившие массивы значений в стандартную библиотеку C++, вышли из комитета по стандартизации задолго до завершения стандарта. В результате массивы значений используются редко.

По указанным выше причинам и чтобы не увеличивать размер книги еще больше, описание класса `valarray` изложено во вспомогательной главе на веб-странице по адресу <http://www.cppstdlib.com>.

